

0.1. Đầu vào

Table 1: Đầu vào của thuật toán GP

Tham số	Mô tả
project	Đối tượng Project chứa activities, resource types, resource units và precedence constraints
pop_size	Kích thước quần thể — số cá thể trong mỗi thế hệ (mặc định: 70)
g_max	Số thế hệ tối đa (mặc định: 200)
Q	Số kịch bản simulation dùng để đánh giá mỗi cá thể (mặc định: 30)
seed	Hạt giống ngẫu nhiên — dùng <code>random.randint(0, 10⁶)</code> để mỗi lần chạy khác nhau nhưng GP và FSGP cùng xuất phát điểm

0.2. Đầu ra

Table 2: Đầu ra của thuật toán GP (GPResult)

Trường	Mô tả
best	Cá thể tốt nhất: APR tree + RTPR tree
history	Log mỗi thế hệ: best, mean, worst fitness; kích thước cây trung bình
total_time_s	Thời gian chạy tổng cộng (giây)

0.3. Biểu diễn cá thể

Mỗi cá thể gồm **hai cây biểu thức GP** được đánh giá đồng thời trong simulation:

- **APR tree** (Activity Priority Rule): gán điểm ưu tiên cho từng activity đang chờ — activity có điểm cao nhất được chọn thực hiện trước.
- **RTPR tree** (Resource Transfer Priority Rule): gán điểm ưu tiên khi cần chuyển resource unit giữa các vị trí — unit có điểm cao nhất được chuyển trước.

0.3.1. Tập terminal (lá cây)

Table 3: Tập terminal của APR tree và RTPR tree

APR tree		RTPR tree	
TS	Thời gian xử lý (task time)	TT	Transfer time
DT	Duration	GAP	Khoảng cách release-deadline
SLK	Slack time = deadline – ES – PT	TWC	Total waiting cost
LF	Latest finish	TTV	Total transfer value
ES	Earliest start	RS	Resource score
LS	Latest start	AT	Arrival time
RR	Remaining resource	const	Hằng số thực ngẫu nhiên
SLF	Slack factor		
EF	Earliest finish		
const	Hằng số thực ngẫu nhiên		

0.3.2. Tập hàm (node nội)

$$\mathcal{F} = \{+, -, \times, \div^*, \max, \min, |\cdot|, \text{neg}\}$$

$\div^*$: phép chia được bảo vệ — nếu mẫu = 0 thì trả về 1.

0.4. Hàm fitness

$$f(\text{ind}) = \frac{1}{Q} \sum_{q=1}^Q \text{makespan}(\text{ind}, s_q) \quad (1)$$

trong đó s_q là kịch bản simulation thứ q (thời gian ngẫu nhiên được rút từ phân phối $\mathcal{N}(\mu_i, \sigma_i^2)$). **Cá thể có fitness thấp hơn là tốt hơn.**

0.5. Tham số tiến hóa

Table 4: Tham số tiến hóa của GP

Tham số	Mặc định	Ý nghĩa
p_crossover	0.80	Xác suất áp dụng crossover
p_mutation	0.15	Xác suất áp dụng mutation (thay ngẫu nhiên một cây con)
p_reproduction	0.05	Xác suất copy nguyên cá thể $= 1 - p_{cx} - p_{mut}$
elitism	2	Số cá thể tốt nhất giữ nguyên qua mỗi thế hệ
tournament_k	3	Kích thước tournament selection — nhỏ hơn giúp duy trì diversity
min_depth	2	Độ sâu cây tối thiểu khi khởi tạo
max_depth_init	6	Độ sâu cây tối đa khi khởi tạo (ramped half-and-half)
max_depth_cx	8	Độ sâu tối đa sau crossover — chống bloat (cây phình to)
max_depth_mut	6	Độ sâu tối đa cây con mới trong mutation
scenario_refresh_ε	10	Sinh lại Q scenarios mới mỗi N thế hệ để tránh GP overfit tập cố định

0.6. Luồng chạy

Algorithm 1 GP Baseline

Require: project, config: GPConfig

Ensure: Cá thể tốt nhất best

```

1: Sinh  $Q$  kịch bản  $\mathcal{S} = \{s_1, \dots, s_Q\}$ 
2: Khởi tạo quần thể  $P \leftarrow \text{init\_population}(\text{pop\_size})$ 
3: Evaluate  $P$  trên  $\mathcal{S}$ ;  $\text{best} \leftarrow \arg \min_{p \in P} f(p)$ 
4: for  $g = 1$  to  $g_{\max}$  do
5:   if  $g \bmod \text{scenario\_refresh\_every} = 0$  then
6:      $\mathcal{S} \leftarrow$  sinh lại  $Q$  kịch bản mới
7:     Re-evaluate toàn bộ  $P$  trên  $\mathcal{S}$  mới
8:   end if
9:    $P' \leftarrow \emptyset$ 
10:  Thêm top-elitism vào  $P'$  ▷ Elitism
11:  while  $|P'| < \text{pop\_size}$  do
12:     $r \sim \mathcal{U}(0, 1)$ 
13:    if  $r < p_{cx}$  then
14:       $p_1, p_2 \leftarrow \text{tournament}(P, k)$ ;  $c_1, c_2 \leftarrow \text{crossover}(p_1, p_2)$ 
15:      Thêm  $c_1, c_2$  vào  $P'$ 
16:    else if  $r < p_{cx} + p_{mut}$  then
17:       $p \leftarrow \text{tournament}(P, k)$ ;  $m \leftarrow \text{mutate}(p)$ 
18:      Thêm  $m$  vào  $P'$ 
19:    else
20:       $p \leftarrow \text{tournament}(P, k)$ ; thêm  $\text{copy}(p)$  vào  $P'$ 
21:    end if
22:  end while
23:   $P \leftarrow P'[: \text{pop\_size}]$ 
24:  Evaluate  $P$  trên  $\mathcal{S}$ 
25:  if  $\min_{p \in P} f(p) < f(\text{best})$  then
26:     $\text{best} \leftarrow \arg \min_{p \in P} f(p)$ 
27:  end if
28: end for
29: return best

```

1. FSGP — Fitness Surrogate-Assisted GP

FSGP mở rộng GP bằng cách thêm một mô hình **Random Forest** (RF) làm *surrogate* — dự đoán nhanh fitness của con cái trước khi quyết định có simulate thực hay không. Mục tiêu: **tiết kiệm số lần simulate** (tốn kém) trong khi vẫn duy trì chất lượng tiến hóa.

1.1. Đầu vào

Giống GP, cộng thêm các tham số surrogate:

Table 5: Tham số surrogate của FSGP (FSGPConfig)

Tham số	Mặc định	Ý nghĩa
n_warmup	100	Số mẫu tối thiểu trong archive trước khi RF bắt đầu lọc (giai đoạn khởi động)
threshold_ratio	0.7	Lọc cá thể dự đoán tệ hơn $f^* + 0.7 \times (f_{\max} - f^*)$
rf_std_ratio	0.3	Chỉ lọc khi $\sigma_{\text{RF}} < 0.3 \times (f_{\max} - f^*)$ — kiểm tra độ tự tin RF
rf_n_trees	30	Số cây trong Random Forest
rf_max_depth	5	Độ sâu tối đa mỗi cây trong RF
max_archive	300	Sliding window — chỉ giữ 300 mẫu gần nhất trong archive
rf_retrain_every	5	Train lại RF mỗi 5 thế hệ (không phải mỗi thế hệ)

1.2. Đầu ra

Table 6: Đầu ra của FSGP (FSGPResult)

Trường	Mô tả
best	Cá thể tốt nhất: APR tree + RTPR tree
history	Log mỗi thế hệ, thêm n_replaced (số cá thể RF lọc) và rf_active
total_time_s	Thời gian chạy tổng cộng (giây)
n_sim_saved	Tổng số simulate tiết kiệm được nhờ RF lọc
n_sim_total	Tổng số simulate thực tế đã chạy

1.3. Vector đặc trưng (17 chiều)

Mỗi cá thể được ánh xạ thành vector $\mathbf{x} \in \mathbb{R}^{17}$ để RF học — không dùng cây GP trực tiếp:

$$\mathbf{x}(\text{ind}) = \left[\underbrace{d_A, s_A, n_A^c, \bar{c}_A, c_A^{\max}, n_A^f, \rho_A^b, \rho_A^k}_{8 \text{ đặc trưng APR}}, \underbrace{d_R, s_R, n_R^c, \bar{c}_R, c_R^{\max}, n_R^f, \rho_R^b, \rho_R^k}_{8 \text{ đặc trưng RTPR}}, \underbrace{d_A \cdot d_R}_{\text{tương tác}} \right] \quad (2)$$

Table 7: Ý nghĩa 17 đặc trưng

#	Ký hiệu	Ý nghĩa
0	d_A	Độ sâu APR tree
1	s_A	Tổng số node APR tree
2	n_A^c	Số node hằng số trong APR tree
3	$\bar{c}_A$	Trung bình giá trị các hằng số APR
4	$c_A^{\max}$	Hằng số lớn nhất trong APR tree
5	n_A^f	Số node hàm (function nodes) trong APR
6	ρ_A^b	Tỉ lệ binary op / tổng ops trong APR
7	ρ_A^k	Tỉ lệ terminal quan trọng SLK, LF / tổng terminal biến APR
8–15	$d_R, \dots, \rho_R^k$	8 đặc trưng tương tự cho RTPR tree (terminal quan trọng: TT, GAP)
16	$d_A \cdot d_R$	Tương tác cấu trúc hai cây

1.4. Random Forest surrogate

1.4.1. Cấu trúc

RF gồm $T = 30$ cây hồi quy CART, mỗi cây được train độc lập trên bootstrap sample của archive:

- Mỗi cây xét ngẫu nhiên $\lfloor \sqrt{17} \rfloor = 4$ đặc trưng tại mỗi node — tạo đa dạng giữa các cây.
- Predict:** trả về cặp $(\hat{\mu}, \hat{\sigma})$:

$$\hat{\mu}(\mathbf{x}) = \frac{1}{T} \sum_{t=1}^T h_t(\mathbf{x}), \quad \hat{\sigma}(\mathbf{x}) = \sqrt{\frac{1}{T} \sum_{t=1}^T (h_t(\mathbf{x}) - \hat{\mu}(\mathbf{x}))^2}$$

$\hat{\sigma}$ cao $\Rightarrow$ các cây bất đồng $\Rightarrow$ RF không tự tin $\Rightarrow$ **không lọc**.

1.4.2. Tìm ngưỡng tốt nhất — $O(n \log n)$ bằng prefix sum

Thay vì duyệt $O(n^2)$, tại mỗi feature f_i :

1. Sắp xếp các mẫu theo x_{f_i} : $O(n \log n)$
2. Xây prefix sum: $S_k = \sum_{j=1}^k y_{\pi(j)}$, $Q_k = \sum_{j=1}^k y_{\pi(j)}^2$
3. Tại điểm chia ℓ , MSE tổng tính trong $O(1)$:

$$\text{MSE}(\ell) = \underbrace{\left(Q_\ell - \frac{S_\ell^2}{\ell}\right)}_{\text{MSE trái} \times n_L} + \underbrace{\left((Q_n - Q_\ell) - \frac{(S_n - S_\ell)^2}{n - \ell}\right)}_{\text{MSE phải} \times n_R} \quad (3)$$

Tổng độ phức tạp: $O(n \log n)$ thay vì $O(n^2)$ per feature per node.

1.5. Điều kiện lọc cá thể

Một con cái bị loại (revert về cha/mẹ) khi **đồng thời** thỏa mãn cả hai điều kiện:

$$\hat{\mu}(\mathbf{x}) > f^* + \alpha \cdot (f_{\max} - f^*) \quad \text{và} \quad \hat{\sigma}(\mathbf{x}) < \beta \cdot (f_{\max} - f^*) \quad (4)$$

trong đó:

- $f^* = \min(\text{archive})$, $f_{\max} = \max(\text{archive})$
- $\alpha = \text{threshold_ratio} = 0.7$ — ngưỡng chất lượng
- $\beta = \text{rf_std_ratio} = 0.3$ — ngưỡng tự tin

Nếu chỉ một điều kiện thỏa mãn $\Rightarrow$ giữ nguyên, vẫn simulate. Thiết kế này tránh RF lọc nhầm cá thể tốt khi còn bất định.

1.6. Luồng chạy FSGP

Algorithm 2 FSGP — Fitness Surrogate-Assisted GP

Require: project, config: FSGPConfig

Ensure: Cá thể tốt nhất best

```

1: Sinh  $Q$  kích bản  $\mathcal{S}$ 
2: Khởi tạo quần thể  $P$ ; evaluate toàn bộ
3:  $\mathcal{A}_X \leftarrow [\mathbf{x}(p) \mid p \in P]$ ;  $\mathcal{A}_y \leftarrow [f(p) \mid p \in P]$  ▷ Khởi tạo archive
4:  $\text{RF} \leftarrow \text{None}$ 
5: for  $g = 1$  to  $g_{\max}$  do
6:   if  $g \bmod \text{scenario\_refresh\_every} = 0$  then
7:      $\mathcal{S} \leftarrow$  sinh lại  $Q$  kích bản mới
8:     Re-evaluate  $P$ ; cập nhật  $\mathcal{A}$ ; trim về  $|\mathcal{A}| \leq \text{max\_archive}$ 
9:   end if
10:  Sinh con cái  $C$  bằng crossover/mutation/reproduction; ghi nhớ cha/mẹ
11:  if  $|\mathcal{A}_y| \geq \text{n\_warmup}$  then
12:    if  $\text{RF} = \text{None}$  or  $g \bmod \text{rf\_retrain\_every} = 0$  then
13:       $\text{RF} \leftarrow \text{train}(\mathcal{A}_X, \mathcal{A}_y)$ 
14:    end if
15:    for  $c \in C$  do
16:       $(\hat{\mu}, \hat{\sigma}) \leftarrow \text{RF.predict}(\mathbf{x}(c))$ 
17:      if  $\hat{\mu} > f^* + \alpha(f_{\max} - f^*)$  and  $\hat{\sigma} < \beta(f_{\max} - f^*)$  then
18:         $c \leftarrow \text{copy}(\text{parent}[c])$  ▷ Revert, không simulate
19:      end if
20:    end for
21:  end if
22:  for  $c \in C$  with  $f(c) = \infty$  do
23:    Evaluate  $c$  trên  $\mathcal{S}$ 
24:     $\mathcal{A}_X \mathrel{+}= [\mathbf{x}(c)]$ ;  $\mathcal{A}_y \mathrel{+}= [f(c)]$ 
25:  end for
26:  Trim:  $\mathcal{A} \leftarrow \mathcal{A}[-\text{max\_archive} : ]$ 
27:   $P \leftarrow \text{sort}(P \cup C)[ : \text{pop\_size}]$ 
28:  Cập nhật best
29: end for
30: return best

```

1.7. So sánh GP vs FSGP

Table 8: So sánh GP Baseline và FSGP

Tiêu chí	GP Baseline	FSGP
Simulate/thế hệ	Tất cả con cái	Chỉ con cái qua lọc RF
Overhead thêm	Không	Train RF + extract features
Chất lượng	Chuẩn	Tốt hơn nếu RF lọc đúng
Rủi ro	Không	RF lọc nhầm → mất hướng tốt
Thời gian	Chậm hơn	Nhanh hơn nếu % tiết kiệm > 10%
Diversity	Giống nhau (tournament_k= 3)	

Lưu ý: FSGP chỉ vượt GP khi RF đủ chính xác để lọc đúng hướng. Nếu % tiết kiệm thấp (< 5%) hoặc $best\ FSGP > best\ GP$, cân nhắc tăng n_warmup hoặc điều chỉnh $threshold_ratio / rf_std_ratio$.